Threat Model — Attribution Integrity Benchmark (AEGIS-AT)

> **How to use this file:** This is your June gate. Fill every section *before* writing attack code. The prompts in `>` blockquotes are questions for you to answer in your own words — delete each prompt once you've replaced it with your answer. The answers are the part you own and must be able to defend line-by-line. AI can scaffold this file; it cannot decide these answers for you.

---

1. System under test

> Describe the system in 3–4 sentences. What are the components (orchestrator, Subagent A, Subagent B, one tool)? What is each allowed to do? Keep it minimal — the smallest system that makes sibling impersonation possible.

Contain impersonates Enrich (high-privilege attributed as low)

- models hiding a consequential containment action behind a benign-looking agent, so it slips past audit.
- This is the "covering tracks" scenario.

Overview. A minimal SOC alert-triage pipeline. A human analyst authenticates and issues a request to a triage orchestrator. The orchestrator delegates work to two sibling subagents — Agent-Enrich and Agent-Contain — by minting each a scoped delegation token (RFC 8693 act claim). Both subagents can call one shared tool, siem_action, which records the identity of the calling agent in the action log. Agent-Contain holds higher-consequence permissions than Agent-Enrich; this asymmetry is what makes misattribution between the two security-relevant rather than cosmetic. The system is deliberately the smallest configuration in which sibling impersonation is possible: two siblings (so impersonation has a target) and one shared tool (so the attack has exactly one degree of freedom — A acted, the log says B).

Components: - Human principal — The SOC analyst. Authenticates once (OAuth2/OIDC), originates the task, and is the root of every delegation chain. Holds the superset of
authority; delegates a narrowed slice downward. Does not act on tools directly.

- Orchestrator — Receives the analyst's request, decomposes it, and mints scoped delegation tokens for the two subagents via token exchange. Each token's act claim nests back to the analyst. The orchestrator is the delegating authority; it does not call siem_action itself.

⟨Critical call: is the orchestrator in or out of the attacker's reach? If the attack mechanism is confused-deputy via the orchestrator, the orchestrator is part of the attack surface; if it's token reuse or scope spoofing at the subagent layer, the orchestrator is trusted. This decision is really a §5 decision, but it determines whether you describe the orchestrator as trusted here.⟩ → Decided: - Trusted: The orchestrator must be inside the attackers reach for the sibling-impersonation attack to be realistic and forensically
interesting

- Subagent A — Enrich — the lower-consequence sibling. Job: read-only context gathering (e.g., pull alert metadata, enrich indicators). Scope: read-only access to siem_action. In the attack, this is the agent whose identity is falsely stamped on Contain's action — the innocent sibling the attacker hides behind.
- Subagent B — the higher-consequence sibling and the true executor in the attack. Job: consequential response actions (e.g., isolate a host, block an IP). Scope: write/ action access via siem_action.

⟨your call: how much do you separate "decide to contain" from "execute containment"? Keeping it to one tool call keeps your one-degree-of-freedom cleanliness; splitting it is more realistic but muddies measurement. I'd keep it single for v1.⟩ → Decided: - Keep it in one Tool Call to keep one-degree-of-freedom cleanliness - splitting can be done in future iterations

- Tool — siem_action — A single SOAR-style endpoint that can execute both read-only queries and write-capable response actions. The command parameter determines the operation, and the delegated token's scope claim determines whether the call is permitted. ⟨your call: does siem_action do both read and action, with scope deciding what's permitted? Or is it one endpoint with a permission check? Simplest defensible version: one tool, scope-gated.⟩ → Decided: One endpoint with a scope-gated permission check is the simplest defensible version for v1 → Decided: Tool Trusts the Token (Option 1) The tool must extract the agent identity solely from the verified act claim in the delegation token. It must not accept a self-reported identity. This is the only design that:

Anchors the attack in the orchestrator, preserving the confused-deputy vector I deliberately chose. Maps directly to real-world delegated token systems (RFC 8693, SPIFFE/SPIRE, HAID) where cryptographic attestation is the identity.

A curve showing how well attribution survives when the orchestrator is under attack — rather than a single "we forgot to check" finding that a reviewer dismisses as a bug.

2. Trust boundaries

Trust boundaries answers two questions a reviewer will absolutely ask: where does a token get checked (so they can see your defenses are at the right places), and where does the attacker's input enter (so they can see your attack model is realistic).

Boundary 1 (principal → orchestrator):

The analyst authenticates via OAuth2/OIDC; the orchestrator verifies the analyst's identity token and uses it as the root of every subsequent delegation chain. Verification: standard OIDC token validation (signature, issuer, expiry). The analyst is a trusted, authenticated insider with legitimate authority. Their task prompt is treated as trusted input.

However, the orchestrator also receives data derived from upstream SIEM alerts as part of its triage workflow. Alert content is definitionally untrusted — it describes activity from outside the security perimeter and routinely contains attacker-controlled strings (hostnames, URLs, log lines, email subjects). The attacker does not compromise the analyst, the SIEM, or any upstream system — they only need to cause an alert whose text reaches the Enrich agent, which processes it and passes extracted fields to the orchestrator as part of the triage workflow. This is the attacker's sole injection point.

Real-world precedent for this vector:

The Cline February 2026 compromise (crafted GitHub issue title flowing into a triage agent); Log4Shell CVE-2021-44228 (crafted string in a log entry triggering privileged action on the processing server); Splunk XSS CVE-2017-5607 (crafted field in an indexed event executing in the consumer's context);

ELK stack log injection via HTTP User-Agent headers. The original confused-deputy lineage traces to Hardy (1988). Alert-as-injection-vector is a decade-old, well-documented pattern, not a novel claim.

Boundary 2 (orchestrator → subagent — token minting): The orchestrator mints scoped delegation tokens for each subagent via RFC 8693 token exchange, narrowing scope and nesting the act claim back to the analyst principal. Verification at this boundary has two distinct components, and the attack exploits their gap:

Cryptographic verification (sound and assumed working): the orchestrator's own identity is verified by the authorization server before it is permitted to exchange tokens. Issued tokens carry valid signatures, correct expiry, and well-formed nested act claims.

The authorization server's signing key is not within the attacker's reach (per §3). Every token minted at this boundary is cryptographically valid.

Decisional verification (the slack):

The orchestrator is an RFC 8693 token-exchange endpoint: it receives a token-exchange request from a subagent (with subject_token, actor_token, requested scope, and audience), validates it per RFC 8693 §2.1, and mints the new token per §4.1 / Appendix A.2.5. The orchestrator does NOT make routing decisions based on alert content; the requesting subagent specifies the exchange parameters. This separation is load-bearing for the §5 finding: the misattribution must be a property of the standard, not of orchestrator routing logic.

The attacker influence at this boundary is therefore upstream — the attacker shapes the alert; the alert shapes Enrich's output; Enrich's output shapes the exchange

request Enrich submits. The orchestrator validates and mints without inspecting alert content.

The attacker cannot forge a token, but they can manipulate Enrich (via alert content) into submitting an exchange request that the orchestrator will honestly honor — yielding a token that names the wrong sibling as the actor. This is the attack boundary. Boundaries 1, 3, 4, and 5 are described so the reviewer can see they are verified properly and are not where the attack lives. Boundary 2 is where verified-but-influenceable upstream input decides identity-bearing tokens, and the slack between "verified mechanism" and "influenceable upstream request" is the gap the benchmark measures.

Boundary 3 (subagent → tool — token presentation and verification):

Either subagent invokes siem_action(command, …) by presenting its delegation token. The tool performs full cryptographic verification before executing the requested command:

- Signature verification: the token's signature is validated against the authorization server's public key. Forged tokens are rejected.
- Expiry and freshness check: the token's exp claim is validated against the current time. Expired tokens are rejected.
- Chain integrity walk: the nested act claim is traversed from innermost to outermost, confirming each hop is structurally well-formed and terminates at the analyst principal. Malformed or unrooted chains are rejected.
- Scope gate: the token's scope claim is checked against the requested command. A token with siem:read invoking isolate_host is rejected; a token with siem:write invoking it is permitted.
- Identity resolution (per §1's locked decision): the calling agent's identity is read solely from the most-recent actor in the delegation chain (the top-level act.sub). No self-reported identity field is accepted. This is the identity that will be written to the action log as the claimed actor.

Verification at this boundary is assumed working and is not where the attack lives. On the adversarial path, the token presented at Boundary 3 was minted at Boundary 2 — it carries a valid signature, current expiry, well-formed chain, scope appropriate to the requested command, and a current actor (top-level act.sub) naming a legitimate sibling. Every check above passes. The tool then writes the claimed actor (read from the verified act claim) to the log, faithfully recording what the token says. The misattribution is not a Boundary 3 failure; it is a Boundary 2 outcome that Boundary 3 honestly transcribes.

Why this boundary matters for the result: a reviewer's first reflex on seeing the attack succeed is "the tool must have skipped a check." This boundary exists to foreclose that reading. Every standard check is named and assumed to pass. The attack is not a verification gap at the tool; it is a decisional gap at the orchestrator, faithfully propagated by a correctly-functioning tool.

Boundary 4 (tool → action log — claimed-actor write):

After Boundary 3's verification completes and the requested command executes, the tool writes a record to the action log: {claimed_actor, claimed_scope, claimed_principal_chain, command, target, timestamp, token_chain_summary}. The claimed_actor field is the identity resolved at Boundary 3 — i.e., the current actor (top-level act.sub) of the verified act claim. The tool does not have access to ground truth (it cannot independently verify which agent process actually originated the call) and so writes what the token says, faithfully.

Verification at this boundary depends on the active baseline:

- Baselines 1–3 (no tamper-evidence): the log is a regular append-only file. Entries are correct at write time but the log itself carries no integrity proof. An adversary with later access to the log could in principle modify, delete, or reorder entries; this is out of scope for v1's threat model (the attacker's capability is alert-content control, per §3), but it means the log alone is not a forensic record.
- Baseline 4 (tamper-evident) — *design; attribution-only in v1*: the intended configuration signs and hash-chains each entry — each entry includes a hash of the previous entry, and the chain is signed by a dedicated logger key not held by any agent or the orchestrator — so post-hoc tampering becomes cryptographically detectable and the log is a forensic record. **In v1 the hash-chaining module is not implemented:** B4 reuses B3's claimed-actor resolution, so its *attribution* equals B3 by construction. Testing log *integrity* (a separate metric from attribution) requires the real hash-chained module and is named as future work. This does not affect the v1 attribution result — the wrong actor is committed upstream at minting, before any logging layer sees the entry.

What this boundary does and does not do in the attack: the attack does not target Boundary 4 — the claimed actor written here is already wrong by the time it arrives, having been corrupted at Boundary 2 and faithfully transcribed by Boundary 3. The log honestly records the misattribution.

Boundary 4 is therefore not where the attack lives, but it is where a defense layer (Baseline 4's tamper-evidence) operates. Tamper-evidence does not detect the original misattribution — the wrong identity was committed before the log ever saw the entry — but it does prevent post-hoc rewriting of the (wrong) record. The benchmark's AIS curve measures whether Baseline 4 closes the attribution gap on its own (hypothesis: it does not, because the wrong identity is established earlier).

Why explicitly naming this matters for the result: a reviewer reading the AIS curve will see Baseline 4 underperform expectations on this attack. Stating here — in the threat model, before any results — that Boundary 4's defense layer cannot catch upstream misattribution pre-empts the response "your tamper-evident log is broken." It is working correctly; it is just not the right defense for this attack class. That negative result is itself part of the contribution.

Boundary 5 (system → harness — ground-truth recorder):

The harness maintains a ground-truth recorder that observes, for every tool invocation, the true originating agent process — independent of any claim the agent or the orchestrator makes.

For each siem_action call, the recorder writes {true_actor, true_scope, true_principal_chain, command, target, timestamp} to a separate ground-truth store.

The AIS scorer compares this record to the corresponding action-log entry written at Boundary 4; a mismatch on actor, scope, or principal_chain (per the metric definition in §4) counts as an attribution defect.

Verification at this boundary: The recorder must be independent of the attacker's reach. This independence is provided by construction along three axes:

1. Process boundary:

the recorder runs in the harness process, distinct from the agent processes. It instruments tool invocations at the harness layer (e.g., by wrapping siem_action in the test harness) so it observes the calling agent's process identity before any agent-supplied data influences the observation. The attacker's capability — alert-content control, per §3 — cannot cross the process boundary into the harness.

1. Credential isolation:

the ground-truth store uses credentials issued only to the harness. Neither the orchestrator, nor either subagent, nor the auth server holds credentials that can write to the ground-truth store. A compromise of any in-system component does not yield ground-truth write access.

1. Causal precedence:

the recorder writes ground truth before the agent's tool call reaches the verification logic at Boundary 3. The recorder's observation is therefore causally upstream of any attack effect at Boundary 2 or Boundary 3.

What this boundary does and does not do: Boundary 5 is not a defense and is not part of the system under test. It is the measurement instrument. Its job is to make the AIS metric meaningful by guaranteeing that "claimed attribution" and "true attribution" come from causally independent sources. If this boundary's independence is violated, the benchmark measures nothing.

Why this matters explicitly for the result: §8 will face the question "how do you know your ground truth is actually ground truth?" The answer lives here: process boundary, credential isolation, causal precedence. A reviewer who accepts those three properties has to accept the AIS measurement as valid. A reviewer who challenges them is challenging the construction of the experiment, not the result — and the construction is described, defensible, and reproducible.

The attack boundary — where sibling impersonation actually occurs:

The architecture has no direct communication path between Enrich and Contain. They do not share tokens, they do not call each other, and they do not share a tool namespace beyond siem_action.

All impersonation must therefore route through a token minted with the wrong act claim at Boundary 2. That is the attack boundary — the only point in the system where an adversarial input can cause one sibling's identity to be attached to the other

sibling's action. Boundaries 1, 3, 4, and 5 are described so the reviewer can see they are verified properly and are not where the attack lives.

Boundary 2 is where verified-but-influenceable upstream input decides identity-bearing tokens, and the slack between "verified mechanism" and "influenceable upstream request" is the gap the benchmark measures.

---

3. Adversary model

The adversary is modeled at the gray-box knowledge level: they know the kind of system they are attacking and the kind of decisions it makes, but not its precise prompts, extraction code, or internal scope strings. This matches the knowledge profile of every real-world precedent cited in §2 (Cline February 2026; Log4Shell; Splunk XSS; Salesloft Drift; ELK injection). The system's defenses are not assumed secret — only its specific implementation details.

What the adversary CAN do:

- Control the text content of one or more SIEM alerts that will be processed by the Enrich agent. This includes any field of the alert that originates from outside the security perimeter: hostnames, URLs, file paths, log lines, email subjects, command-line strings, user-agent headers, and other free-text fields routinely populated from external observation.
- Cause those alerts to enter the SIEM through normal channels (i.e., the attacker does not need to bypass the SIEM's ingestion controls; they only need to cause activity that legitimately produces an alert whose content they shaped).
- Repeat the attack — there is no rate limit assumed in the threat model; the attacker can make many attempts.

What the adversary CANNOT do:

- Compromise the analyst, the analyst's session, or the analyst's task prompt.
- Compromise the SIEM, upstream sensors, or any other system in the data path before Enrich.
- Forge or modify delegation tokens. The authorization server's signing key is out of reach (per §2 Boundary 2's cryptographic verification).
- Read or write the ground-truth recorder. The harness process and its credentials are out of reach (per §2 Boundary 5's three independence axes).
- Tamper with the action log after it is written. (Out of scope for v1 — see backlog in §7.)
- Compromise the orchestrator, either subagent, or the tool as processes. The attacker influences these components only by controlling the data they consume; they do not execute code inside them.

What the adversary KNOWS:

- The system uses a multi-agent SOC triage architecture with an orchestrator and at least two sibling agents differentiated by scope/consequence.

- Alert content is parsed by an early-stage agent and influences downstream routing decisions, including which agent is invoked for which actions.
- Some form of cryptographic delegation (RFC 8693 or equivalent) is in use; tokens carry identity and scope claims that cannot be directly forged.
- Tool calls are recorded in an audit log; some baselines may make this log tamper-evident.

The adversary does NOT know:

- The exact text of the orchestrator's prompt or system instructions.
- The exact field-extraction logic in Enrich.
- The exact scope strings, agent names, or tool command names as they appear in code. (They may have plausible guesses derived from public SOC/SOAR documentation, but they have no oracle to confirm them.)
- Which baseline defense configuration is active during any given attempt.
- The internal state of the ground-truth recorder.

This knowledge profile matches an attacker who has read public documentation about agent-based SOC tooling, observed the system's external behavior from afar (e.g., as an upstream actor whose alerts get triaged), or reverse-engineered the system from observable side effects — but who does not have insider access to the codebase or runtime state.

Adversary's goal: Cause an action truly performed by Agent-Contain to be attributed to Agent-Enrich in the action log, such that a SOC analyst reviewing the log sees no record of the high-consequence action having been taken by the high-consequence agent.

The attack succeeds when the AIS scorer (per §4) records a mismatch on actor, scope, or principal_chain for an action that the ground-truth recorder confirms was performed by Contain.

The attack is "covering tracks," not "privilege escalation" — the attacker is not trying to gain new authority, only to hide who exercised authority that already existed.

What the adversary does not need:

- zero-day vulnerabilities
- model jailbreaks
- instruction-override directives
- adversarially optimized embeddings

The threat is structural and exploits documented confused-deputy dynamics (Hardy 1988; Cline 2026; CSA AI Safety Initiative confused-deputy research note, March 2026). The attacker's craft is in shaping alert content such that legitimate parsing and routing produce a token whose act claim names the wrong sibling.

4. What "true attribution" means (the ground-truth definition)

This section defines ground truth and the AIS metric formally. Everything downstream — the scorer implementation, the results section, the validity argument in §8 — depends on these definitions being precise.

Ground-truth schema.

For each tool invocation, the harness's ground-truth recorder (per §2 Boundary 5) records the tuple:

{true_actor, true_scope, true_principal_chain, command, target, timestamp} Field semantics:

true_actor:
the agent identity that actually executed the call. Determined by the harness from the agent process that invoked siem_action, not from any token or self-reported field. Values: agent:enrich or agent:contain.

true_scope:
the scope the action genuinely required, as a function of the command parameter. Determined by the harness's static mapping of commands to required scopes (read commands → siem:read; action commands → siem:write).

true_principal_chain:
the delegation path the true actor was legitimately operating under, as an ordered list from immediate actor outward to the human principal: [true_actor, "human:analyst"]. This is a 2-hop chain. The orchestrator does NOT appear in it: the orchestrator is a stateless token-exchange endpoint (per §2 Boundary 2), not a principal that holds a delegation token, so RFC 8693's act claim records no hop for it. The chain shape here therefore matches what a spec-compliant single-hop delegation token produces on the claimed side (the tool's actor_chain walk yields [current_actor, principal]). Determined by the harness from the agent's legitimate context at invocation time, independent of any act claim presented to the tool.

command, target, timestamp : descriptive fields recording what was invoked, against what target, at what time. Not part of attribution but required for matching ground-truth records to claimed-actor records at Boundary 4.

The corresponding claimed-actor record at Boundary 4 has the parallel shape:

{claimed_actor, claimed_scope, claimed_principal_chain, command, target, timestamp, token_chain_summary}

with each claimed_* field derived from the verified act claim of the presented delegation token (per §2 Boundary 3's identity resolution). Records are matched between the two stores using (command, target, timestamp).

The AIS metric.

For a single adversarial action a, define the indicator:

```
is_correct(a) = 1   if   claimed_actor(a)             == true_actor(a)
                    and  claimed_scope(a)             == true_scope(a)
                    and  claimed_principal_chain(a)   == true_principal_chain(a)
              = 0   otherwise
```

Comparison is strict: all three fields must match exactly.

Comparison of principal_chain is ordered-list equality: the two lists must have the same length, the same members, in the same order. Any deviation — including a permutation, a missing hop, or an inserted hop — is a defect.

Where no delegation chain exists — an opaque per-agent credential at Baselines 1–2 — principal_chain is None and is scored only when present: None on both the claimed and true sides is a match, while None against a populated chain is a defect. The ordered-list equality above governs once a delegation chain exists (Baselines 3–4).

The Attribution Integrity Score for a baseline configuration B is then:

AIS(B) = (Σ_{a ∈ A(B)} is_correct(a)) / |A(B)|

where A(B) is the set of adversarial actions executed under baseline B — that is, tool calls that the attack actually influenced. Non-adversarial calls (calls made during attack setup, baseline calibration, or unattacked runs) are not included in the denominator. AIS is reported per baseline, not aggregated; the curve across baselines is the result.

Derived reporting metrics.

Two derived quantities are reported alongside AIS to support forensic interpretation:

- Defect breakdown. 1. For each adversarial action where is_correct = 0, record which of the three fields mismatched. Reporting the distribution of defect types across a baseline shows whether the attack breaks attribution uniformly across all three fields or concentrates on one. This is the diagnostic signal for §6 (defense baselines) — it tells you which field a given defense layer actually protects.

1. Hold rate at each defense layer. For each baseline transition (1→2, 2→3, 3→4), report the marginal improvement in AIS. This isolates the contribution of each defense layer to the curve and is what the writeup will reference when claiming "Baseline 3 closes most of the gap" or "Baseline 4 closes very little."

Baseline-aware reporting.

The active baseline is recorded in every ground-truth record so that AIS can be computed per baseline without re-running. This is an implementation detail but worth stating: the harness writes baseline_id into each ground-truth record alongside the schema fields above, so a single experimental run can produce all four baselines' AIS values if configured to sweep.

Stability and sample size.

The v1 attack is **categorical, not stochastic**: under scripted, deterministic agents the misattribution succeeds *by construction* on every adversarial action, so the finding is a curve *shape*, not a frequency estimate. The harness establishes this with verify_deterministic(), which proves each baseline yields byte-identical records across repeated runs — so a single canonical execution per baseline is sufficient, and confidence intervals are **degenerate by design** (AIS ∈ {0, 1} per baseline, not a sample proportion). The originally-planned stochastic sweep — a probabilistic policy under which AIS becomes a real attack-*frequency* estimate, scored over N independent actions with 95% Wilson intervals (starting at N = 100 per baseline, increased if intervals are wide) — is **future work**, pairing with the v2 expanded denominator in §8.8. The N/Wilson design is retained here as the v2 measurement plan, not the v1 method.

What this section does NOT define.

This section defines what attribution correctness means and how to score it. It does not define:

- The specific attack mechanism that causes adversarial actions to occur — that is §5.
- The defense layers that each baseline applies — that is §6.
- The validity of the ground-truth recorder itself — that was argued in §2 Boundary 5 and will be defended in §8.

---

5. The attack mechanism

This section specifies the single attack mechanism in scope for v1: delegation-chain misattribution via re-delegation (Path B). The mechanism exploits a structural property of RFC 8693 delegation — that the act claim records the agent who requested a delegated token, not the agent who ultimately executes the action — in a multi-agent setting where the requesting and executing agents differ.

The structural property being exploited.

Under RFC 8693 token exchange, when agent X requests a delegated token to perform an action on behalf of principal P, the resulting token’s act claim records X as the actor in the delegation chain. This is correct and by design: the chain answers “on whose authority, through which delegating parties.” But it answers a question subtly different from the one an audit log is assumed to answer.

The audit question is “who performed this action?” The delegation chain records “who requested the authority under which this action was performed.” In a single-agent setting these coincide. In a multi-agent setting where one agent requests a delegated capability that another agent executes, they diverge — and the divergence is invisible to every cryptographic check, because nothing was forged or malformed.

Normal operation (no attack).

Agents act on parsed alert content to decide when a response is warranted, as real SOAR pipelines do (severity, asset criticality, alert type, and affected-host classification routinely drive whether an automated response fires). Enrich makes this escalation decision and submits the corresponding RFC 8693 token-exchange request to the orchestrator, specifying its own token as the actor_token and the required scope. The orchestrator's role is to validate that request per §2.1 and mint the appropriately-scoped, correctly-nested token per §4.1 / A.2.5 — without inspecting alert content. The orchestrator does not read alert content to construct the act chain — it builds the chain from the presented actor_token (per §2 Boundary 2).

A typical flow:

1. An alert arrives and is processed by Agent-Enrich.
2. Enrich enriches the alert (read-only context gathering under siem:read).
3. Where the enriched alert warrants a response action, Enrich initiates a re-delegation request to the orchestrator for a containment capability, presenting its own delegation token as the actor_token.
4. The orchestrator validates Enrich's token cryptographically (per §2 Boundary 2), confirms Enrich is a legitimate agent entitled to request containment on the analyst's behalf, and performs an RFC 8693 exchange, minting a siem:write-scoped token whose act chain nests Enrich (the requesting agent) → orchestrator → analyst.
5. The containment action executes and is logged.

Every step is legitimate. This flow is the system working as designed.

Token structure under Baselines 3–4 (RFC 8693-compliant). When the orchestrator performs the re-delegation exchange — presenting Enrich's token as the actor_token on behalf of the analyst principal — the issued delegation token follows RFC 8693 §4.1 delegation semantics. Per the spec's delegation example (Appendix A. 2.5), sub carries the principal (on whose behalf the action is taken) and the act claim carries the current actor (the party wielding the delegated authority).

```
sub:   "human:analyst"              ← principal (on whose behalf)
scope: "siem:write"
act: {
  sub: "agent:enrich"               ← current actor (requester/wielder)
}
```

The chain is two hops: the current actor (agent:enrich) and the root principal (human:analyst). The orchestrator does not appear — it minted this token but holds no delegated authority of its own, so RFC 8693's act claim records no hop for it (see the note below). A deeper chain would nest further act claims here as prior actors (informational only, per §4.1), but the v1 single-hop attack produces exactly this two-hop shape. Two properties of this structure are decisive, and both follow directly from RFC 8693 §4.1:

1. The current actor is the requester, not the executor. Per §4.1, "the outermost act claim represents the current actor." Here that is agent:enrich — the agent that

requested the delegated token. The tool resolves claimed_actor from this current actor (per §2 Boundary 3).

2. The executing agent does not appear in the token at all. Agent-Contain — the entity that actually invokes siem_action — is nowhere in the spec-compliant token. There is no claim in the RFC 8693 structure that records "who wielded the token at the resource," distinct from "who was delegated the authority." The spec's own examples (§A.2.3) describe the act subject as "the actor that will wield the security token," implicitly assuming the requester and the wielder are the same entity. In the multi-agent re-delegation pattern (§5), they are not.

The misattribution is therefore not a property of any field being read incorrectly. It is a property of the spec-compliant token having no field that can express the divergence between requester and executor. The token faithfully records everything RFC 8693 defines; the executor's identity is simply not among the things RFC 8693 defines.

A note on the orchestrator's absence from the chain. The orchestrator appears NOWHERE in the act chain either — and for a reason worth stating, because it bears on the central thesis. When this threat model was first drafted, §4's ground-truth schema recorded the chain as [true_actor, "agent:orchestrator", "human:analyst"], a 3-hop list. The intuition was natural: the delegation flowed analyst → orchestrator → enrich, so surely all three belong in the chain. That intuition is wrong about RFC 8693. The orchestrator is a stateless token-exchange endpoint (§2 Boundary 2); it holds no delegation token of its own and is not a delegated principal. RFC 8693's act claim records principals who hold delegated authority, not the endpoints that mint tokens. A spec-compliant token produced by the exchange therefore contains only [current_actor, principal] — a 2-hop chain — and §4's schema was corrected to match. That the benchmark's own author initially modeled a hop the standard does not produce is itself small evidence for the thesis: the gap between how practitioners intuitively reason about delegation chains and what RFC 8693 actually records is exactly the gap that makes this misattribution surface in real systems. The "→ orchestrator →" phrasing used elsewhere in this section is conceptual — it describes the flow of the exchange request, not the literal nesting of the minted token.

The attack.

The attacker's capability (per §3) is to control the content of an alert that Enrich processes. The attack does not require fooling Enrich into a wrong decision, embedding instructions, or causing any component to misbehave. It requires only that the attacker cause an alert that genuinely warrants containment to flow through the Enrich → re-delegation path.

When that happens:

1. The alert legitimately contains containment-warranting indicators (e.g., a critical-listed source IP, a high-severity signature). The attacker shaped these — but they are real indicators that correctly trigger a containment response. Enrich is not deceived; it makes the right call.
2. Enrich correctly initiates a re-delegation request for a containment action, presenting its own token as the actor_token.

3. The orchestrator correctly mints a siem:write token whose act chain nests Enrich as the requesting agent.
4. The containment action is executed by Agent-Contain (the agent that actually holds and runs the containment capability — the true executor).
5. The tool verifies the token (Boundary 3: signature, expiry, chain, scope all pass), reads the current actor — the outermost act.sub, per RFC 8693 §4.1 — which is Enrich, and records claimed_actor = enrich in the action log (Boundary 4).
6. The ground-truth recorder (Boundary 5) records true_actor = contain, observing the agent process that actually executed the call.
7. The AIS scorer (per §4) compares the two and flags an actor mismatch: claimed Enrich, true Contain. Attribution has failed.

A SOC analyst reviewing the log sees the high-consequence containment action attributed to the read-only enrichment agent. The true executor — Contain — appears nowhere in the record of having taken the action. This is the "covering tracks" outcome from §3: the consequential action is hidden behind a benign sibling's identity.

Why this survives every objection.

1. "The orchestrator must have a bug." No. It validated tokens cryptographically and built the chain from the presented actor_token, exactly as RFC 8693 specifies. The act chain correctly records the requesting agent.
2. "Enrich was manipulated / prompt-injected." No. Enrich made the correct decision — the alert genuinely warranted containment. The result holds even if Enrich is a perfect, unfoolable agent, because the misattribution arises from delegation semantics, not from Enrich's judgment.
3. "The tool skipped a check." No (per §2 Boundary 3). Every check passed. The tool faithfully recorded the current actor from the verified act claim, exactly as RFC 8693 §4.1 mandates.
4. "You just didn't sanitize alert text." No. No component read identity from alert text. The orchestrator built the chain from tokens, not alert content. Alert content's only role was to legitimately trigger a containment-warranting situation.

The attack works because the delegation chain answers "who requested" while the audit log is trusted to answer "who acted," and in the multi-agent re-delegation pattern those are different agents. Every component is correct; the gap is in what the records mean.

What makes an action "adversarial" (for §4's denominator).

An action is adversarial if it is a containment action executed via the Enrich → re-delegation path triggered by attacker-shaped alert content. Per §4, only these actions are counted in the AIS denominator. (Note: this same misattribution can occur in normal operation whenever containment is re-delegated through Enrich — see the framing note below — but the benchmark scopes its denominator to attacker-triggered instances for a clean, attributable measurement.)

Framing note — latent gap vs. adversarial trigger. The misattribution described here is, strictly, a latent property of the re-delegation pattern: it would occur whenever

a containment action is re-delegated through Enrich, attack or no attack. The adversarial framing is that an attacker can deliberately and repeatedly trigger this latent gap by shaping alert content, turning a silent attribution weakness into a controllable “covering tracks” capability. v1 measures the adversarially-triggered case. The observation that the gap also exists in normal operation strengthens the finding (the vulnerability is structural, not merely adversarial) and is developed in §8.

Out of scope for v1 (named so it can’t be called missing).

Direct prompt injection (embedding explicit instructions in alert text). This is an alternative mechanism that fits the gray-box profile, but it conflates the delegation-layer measurement with LLM instruction-following robustness, introducing model-specific confounds. Field-structure-triggered re-delegation isolates the delegation layer from the model’s behavior. Direct prompt injection is named as future work in §8.

Inducing Enrich into an incorrect decision. The v1 attack deliberately relies on Enrich behaving correctly, so the result is independent of Enrich’s robustness. Attacks that manipulate Enrich’s judgment are a distinct mechanism and are out of scope.

The other four sibling-impersonation variants (delegation forgery, scope-attenuation bypass, audit-log tampering, principal laundering) remain backlogged per §7.

6. Defense baselines

The benchmark measures AIS across four defense configurations, applied as config flags over a single codebase (not four separate implementations). Each baseline adds one layer to the previous. The key quantity is not any single baseline’s AIS but the shape of the curve across them — specifically, whether attribution improves monotonically as defenses are added, or whether some defense layer regresses it. For each baseline, two things are stated: what signal the tool uses to determine the claimed actor, and whether that signal tracks the true executor in the Path B re-delegation scenario (§5).

Baseline 1 — Shared service account.

All agents share a single credential. The tool cannot distinguish which sibling is calling, because every call presents the same identity.

Signal read: a single shared identity, identical for all agents. Tracks executor? No — but not by misattributing to the wrong sibling; rather, attribution is undefined. There is no per-agent identity to be right or wrong about. Predicted AIS: ≈ 0.0 (no call can be correctly attributed to a specific sibling). Real-world status: the common-but-wrong status quo (many SOC deployments give agents shared API keys). The baseline quantifies how bad the naive default is.

Baseline 2 — Per-agent identity (no delegation chain).

Each agent holds its own credential. Attribution is determined at authentication time: the tool records the identity of the credential presented by the calling process.

1. Signal read: the authenticating principal at execution time.
2. Tracks executor? Yes. Contain executes the action, so Contain's credential authenticates, so the tool records Contain. Claimed actor = true actor.
3. Predicted AIS: ≈ 1.0. This is the baseline that gets attribution right — not by design sophistication, but because in a per-agent-identity model the executor is the authenticator.
4. Note: authorization still flows through Enrich's request, but attribution is bound to authentication, which tracks the
   executor. The separation of "who was authorized" from "who is recorded acting" is what keeps attribution correct here. This separation is precisely what RFC 8693 removes in Baseline 3.

Baseline 3 — Per-agent identity + delegation across the requester/wielder boundary.

Delegation chains are added. The tool now resolves the claimed actor from the delegation chain's current actor (the top-level act.sub), as mandated by RFC 8693 §4.1: "For the purpose of applying access control policy, the consumer of a token MUST only consider the token's top-level claims and the party identified as the current actor by the act claim. Prior actors identified by any nested act claims are informational only and are not to be considered in access control decisions."

1. Signal read: the current actor in the delegation chain (act.sub) = Enrich, the requester.
2. Tracks executor? No — and the reason is precise. RFC 8693's current actor is the wielder: the party that presents the token at the resource (§A.2.3 calls it "the actor that will wield the security token"; §1.1 frames it as the agent actually taking the action). Here the orchestrator mints the token naming Enrich, the requester — because at mint time it does not yet know which sibling will execute (see §8.5). The token thus asserts Enrich as the wielder. Under unbound bearer tokens, Contain lifts that token and presents it, and nothing in the protocol detects the substitution. The tool, applying §4.1 correctly for the access-control decision, resolves the current actor (Enrich) and the system records it; the executor (Contain) appears in no field the standard defines.
3. Predicted AIS: ≈ 0.0. Attribution is wrong.
4. This is the central result — narrower and more durable than "the spec forces this." The misattribution needs two ingredients, neither sufficient alone: (i) unbound bearer tokens carrying a current-actor claim — RFC 8693 inherits OAuth 2.0's default holder model and does not require sender-constraint, so a token naming Enrich can be presented by Contain; and (ii) a multi-agent hand-off where the wielder differs from the issuer-named actor — the orchestrator names the requester because the executor is undetermined at mint time. §4.1's MUST is scoped to the access-control decision and is silent on audit; the realistic implementation reuses the access-control identity (the current actor) for the audit record, and that identity

is necessarily the named requester. RFC 8693 therefore does not prevent the misattribution and offers no compliant field that records the executor when it differs from the named actor — but neither does it mandate it. Signed delegation regresses attribution relative to Baseline 2 because it introduces a current-actor claim that the unbound-bearer hand-off renders unreliable, where Baseline 2's execution-time authenticator could not be lifted. (The standardized binding hypothesized to prevent the lift — sender-constraint via DPoP / RFC 8705 — is named in §8.10 and deferred to future work.)

Baseline 4 — + Tamper-evident log.

The action log is designed to be made tamper-evident (signed, hash-chained entries under a logger key held by no agent or the orchestrator). **In v1 this module is not built — B4's attribution equals B3 by construction**, and a real tamper-evident log (which would test log *integrity*, a metric separate from attribution) is future work. The analysis below is the prediction that future module would confirm; it does not describe code shipped in v1.

1. Signal read: same as Baseline 3 (current actor in the chain). Tamper-evidence protects the integrity of the recorded entry; it does not change what is recorded.
2. Tracks executor? No. The wrong actor (Enrich) was committed upstream at Baseline 3's resolution step, before the log entry was written. Tamper-evidence cryptographically preserves the wrong record.
3. Predicted AIS: ≈ 0.0 (unchanged from Baseline 3).
4. This confirms the prediction set out in §2 Boundary 4: tamper-evident logging — the second primitive standards bodies emphasize — cannot recover correct attribution, because the misattribution is established before logging. It protects a wrong answer from being altered.

The predicted curve.

Baseline	Configuration	Signal read	Tracks executor?	Predicted AIS
1	Shared account	shared credential (no chain)	undefined	≈ 0.0
2	Per-agent identity	per-agent authenticator (no chain)	yes	≈ 1.0
3	+ delegation across the requester/wielder boundary	delegation current actor (requester)	no	≈ 0.0
4	+ tamper-evident log	delegation current actor (requester)	no	≈ 0.0

The curve is non-monotonic: it rises from Baseline 1 to Baseline 2, then falls at Baseline 3 and stays low at Baseline 4. The headline finding is the drop at Baseline 3 — the two primitives most emphasized for agent non-repudiation (signed delegation chains and tamper-evident logs) do not close the multi-agent attribution gap, and signed

delegation actively opens it relative to simple per-agent identity, by following RFC 8693 §4.1 correctly.

Hypotheses, not results.

These AIS values are pre-registered hypotheses, stated before implementation, to be confirmed or refuted by measurement. If the measured curve differs — for example, if Baseline 3 does not fully collapse, or if Baseline 2 does not reach 1.0 — the discrepancy is itself a finding to investigate, and the threat model commits to reporting it. The defect-breakdown metric (§4) will show which of the three attribution fields (actor, scope, principal_chain) each baseline gets right or wrong. Note that for this attack the actor and principal_chain defects are expected to be correlated (both flag when Enrich occupies the current-actor position); this correlation is a true property of the attack, not a metric artifact, and is reported as such.

Status (post-build): the predicted curve was reproduced deterministically against the real modules; every AIS value is asserted in the test suite against the prediction recorded here before the attack code was written. A contradicted prediction would be reported as a finding, not silenced. See the repository README for build status (test count, gate, reproduction).

---

7. Scope Discipline

This benchmark measures one attribution failure mode, rigorously, rather than surveying many shallowly. This section states precisely what is in scope for v1 and what is deliberately excluded, so that the contribution is not mistaken for either more or less than it is.

In scope (v1).

A single failure mode: sibling misattribution via re-delegation (the Path B mechanism specified in §5). Concretely, the benchmark measures whether the attribution triple {actor, scope, principal_chain} recorded for a containment action matches the true executor, when that action reaches the tool through the Enrich → re-delegation path, across the four defense baselines of §6. This one mode is measured end-to-end: real RFC 8693 token exchange, real per-baseline defenses, an independent ground-truth recorder (§2 Boundary 5), and the AIS metric (§4) with pre-registered hypotheses.

Explicitly out of scope (named so the omissions are deliberate, not gaps). The following are not measured in v1. Each is a legitimate attribution concern; each is excluded for a stated reason, and most are candidates for v2.

Direct prompt injection (embedding explicit instructions in alert text). Excluded because it conflates the delegation-layer measurement with LLM instruction-following robustness, introducing model-specific confounds (§5). The v1 mechanism deliberately isolates the delegation layer from model behavior. Named as future work in §8.

Inducing Enrich into an incorrect decision. Excluded because the v1 attack relies on Enrich behaving correctly, so the result is independent of Enrich’s robustness (§5). Attacks on Enrich’s judgment are a distinct mechanism.

Delegation forgery / token replay. The adversary cannot forge or replay tokens in v1 (the signing key is out of reach, per §3). A weaker key-management model is a separate threat.

Scope-attenuation bypass. Whether an agent can exceed its granted scope is a separate question from whether the correct agent is attributed; v1 holds scope enforcement sound (§2 Boundary 3) and measures attribution only.

Audit-log tampering. The adversary cannot tamper with the log in v1 (§3); Baseline 4 measures whether tamper-evidence helps, but post-hoc log rewriting by a log-capable adversary is out of scope.

Principal laundering (obscuring the human principal at the root of the chain). v1 holds the principal (analyst) correctly rooted and attacks the actor position only.

may_act enforcement (RFC 8693 §4.4). The may_act claim governs authorization to act, not attribution. Since Contain is legitimately authorized in the v1 scenario, may_act does not prevent the attack; whether it offers defense-in-depth value is noted for future work, not measured here.

Why one mode, measured well, beats five gestured at. A benchmark's value is in the trustworthiness of its measurement, not the breadth of its coverage. Measuring one failure mode end-to-end — with an independent ground-truth recorder, spec-compliant token exchange, four real defense baselines, and pre-registered hypotheses — produces a result a reviewer can verify and a practitioner can act on. Surveying five modes shallowly would multiply the surface area for "but did you really measure that correctly?" objections without deepening any single answer. The contribution is a defensible measurement of whether RFC 8693 delegation preserves attribution under sibling impersonation; that claim is strongest when the one mode behind it is airtight. The out-of-scope list above is the v2 roadmap, not a set of excuses.

---

8. Validity threats (pre-empt the reviewer)

A benchmark's value rests on whether its measurement can be trusted. This section lists the ways the result could be wrong or unconvincing, and states the mitigation for each. The first is a genuine limitation that v1 does not fully resolve; it is conceded rather than defeated. The remainder are threats the construction addresses.

1. "You measured one system. Does this generalize?" (The central limitation — conceded.)

This is the deepest objection, and it is partly correct. The non-monotonic AIS curve is demonstrated on one minimal SOC pipeline with one orchestrator design and one re-delegation topology. v1 establishes that the attribution gap exists, is triggerable by a realistic gray-box adversary, and survives the two defenses standards bodies emphasize — in this system. It does not prove the gap holds across all RFC 8693 deployments or all multi-agent delegation topologies. That is a real gap between "shown in one instance" and "true in general," and v1 does not close it.

What makes the result meaningful despite n=1 is why the gap appears. The misattribution is not a quirk of this topology; it follows from RFC 8693 §4.1, which scopes its MUST to the access-control decision (the current actor), reused in practice for the audit record,, combined with the standard's implicit assumption (§A.2.3, §A.2.5) that the current actor and the executor are the same entity. That assumption is topology-independent. Wherever a multi-agent system separates the requester of a delegated capability from its executor, the gap should appear — because the standard provides no field to record the executor when it differs from the requester. v1 measures one instance; the mechanism behind it is structural, which makes generalization likely but unproven. Establishing the gap's prevalence across real delegation architectures is the primary item of future work.

This concession is stated first, and deliberately, because a reviewer will reach for it first; meeting it head-on is more credible than burying it.

1. "The attack only works because your system is a toy."

The system is minimal by design (§1), but minimality is not the same as unrealism. Every structural element maps to a documented real-world pattern: alert-content-as-injection-vector (Cline 2026, Log4Shell, Splunk XSS, ELK injection; §2 Boundary 1), data-driven SOAR routing (§5), and RFC 8693 delegation as the non-repudiation primitive NIST/NCCoE explicitly recommends (Feb 2026). The architecture is the smallest system in which sibling impersonation is possible, not an unrepresentative one — it isolates the one degree of freedom (requester ≠ executor) without confounds. A larger system would add realism but also add confounds that obscure the measurement; v1 trades breadth for a clean causal claim.

1. "Ground truth isn't really independent of the log."

If the ground-truth recorder could be influenced by the same adversarial input that corrupts the log, the AIS measurement would be circular. §2 Boundary 5 establishes independence by construction along three axes: process boundary (the recorder runs in the harness, outside the agent processes the attacker influences), credential isolation (no in-system component holds ground-truth write credentials), and causal precedence (ground truth is recorded before the tool's verification logic runs). The attacker's only capability (alert-content control, per §3) cannot cross any of these. A reviewer who accepts the three axes must accept the measurement; one who challenges them is challenging a described, reproducible construction, not an unstated assumption. Honest sub-limitation: this independence is established by construction and argument, not by formal verification. v1 asserts it; it does not machine-check it.

1. "The baselines aren't a fair comparison."

The four baselines are config flags over a single codebase (§6), not four separately-engineered systems, so differences in AIS cannot be attributed to incidental implementation quality. Each baseline adds exactly one layer to the previous, isolating that layer's marginal effect. The most attackable point — Baseline 2 reaching ≈ 1.0 — rests on a stated, defensible model (attribution binds to the execution-time

authenticator, which is the executor; §6), with the authorization-vs-attribution distinction made explicit so the 1.0 is not an artifact.

1. “This is just a logic bug a careful engineer would fix.”

The honest defense is not that the spec forces the behavior — §4.1’s MUST is access-control-scoped (§6). It is topological. The fix a reviewer reaches for is “name the wielder, not the requester.” But in the topology §1 describes, the orchestrator mints the delegated token and hands it downstream, where routing decides which sibling executes; the wielder is not determined at mint time, so the orchestrator cannot place it in the current-actor claim. Naming the requester is not a careless choice this implementation made — it is what any orchestrator must do when minting precedes execution-routing, which is the realistic multi-agent shape, not an unusual one.

Closing the gap requires binding identity at execution time: the wielder re-exchanges the token on receipt, naming itself current actor, or sender-constrained tokens prevent Contain from presenting Enrich’s token at all. Both work — and both are exactly the execution-identity binding §8.10 names as the layer beyond Baseline 4, deferred to future work. So the objection does not dismiss the result; it identifies the missing layer the result measures the absence of. A “careful engineer” closes this gap only by adding a binding none of the four tested baselines include — which is the finding, not a refutation of it.

1. “The agents’ decisions are scripted, not real LLM behavior.”

In v1, the attack is deliberately designed to be independent of the agent’s reasoning quality: Enrich’s escalation decision is the correct response to a genuinely containment-warranting alert (§5, Reading 2), so the result holds whether the agent is a scripted policy or a frontier model. This is a strength, not a gap — it removes model-specific confounds. But it also means v1 does not measure how the gap interacts with imperfect agent decisions (a manipulated Enrich), which is named out of scope (§7) and left to future work.

1. “Why gray-box and not white-box?”

The gray-box knowledge level (§3) matches every real-world precedent cited (§2) and avoids the Kerckhoffs category error: RFC 8693 runtime prompts are not cryptographic schemes, so white-box exposure of the orchestrator’s prompt would collapse the attack into prompt engineering against a known target, conflating the delegation-layer measurement with model robustness. Gray-box is the level at which the attack is both realistic and reproducible.

1. “Does this gap exist in normal operation, or only under attack?”

The misattribution gap is structural, not merely adversarial.

The §5 attack mechanism describes a path triggered by attacker-shaped alert content. The AIS denominator (§4) is scoped to such attacker-triggered actions to keep the measurement clean and attributable. But the underlying misattribution is a *latent property of the re-delegation pattern itself*: any containment action re-delegated through Enrich produces the same wrong-actor record, attack or no attack. The

attacker does not create the vulnerability; the attacker exploits a property that exists in normal operation.

This framing was noted at the end of §5 and is restated here for emphasis. It strengthens the finding: the vulnerability is not “attacks against AEGIS-AT succeed X% of the time” but “RFC 8693 delegation, applied to multi-agent re-delegation flows, produces misattribution as a latent property of the unbound-bearer hand-off — the spec neither prevents nor mandates it (§4.1’s MUST is access-control-scoped) and offers no field that records the executor. The attack makes the wrongness profitable; the wrongness is intrinsic.”

For v1, the denominator stays scoped to attacker-triggered actions (the cleanest measurement). For v2, an expanded denominator including all re-delegated containment actions would more faithfully measure the structural property.

1. “Your ground-truth recorder uses threads, not processes — is that really independent?”

v1 process-boundary proxy.

The ground-truth recorder (§2 Boundary 5) uses Python’s `threading.current_thread().name` as a proxy for OS process identity. A misbehaving agent that renamed its thread mid-call could spoof ground truth. This is excluded by INV-5 of the project’s invariants (no illegitimate component behavior) and by §3 of this threat model (the adversary controls alert text only, not agent code). The v1 proxy holds within the threat model the project tests. v2 hardening would use `multiprocessing` with `os.getpid()` for a true process boundary.

1. “Sender-constrained tokens would close this — why didn’t you test them?” (The layer beyond Baseline 4 — named, scoped out.)

The gap requires unbound bearer tokens (§6): Contain can present a token minted for Enrich only because nothing binds the token to its holder. Two standardized mechanisms bind it — DPoP (RFC 9449) and mutual-TLS-bound tokens (RFC 8705). Under either, Contain cannot present Enrich’s token and must obtain its own, so the current actor would track the executor. Sender-constraint is effectively a Baseline 5 — the layer hypothesized to recover the curve; whether it does is a question for future work, not a result claimed here. v1 does not implement it, deliberately: the contribution is to show that the primitives standards bodies currently emphasize for agent non-repudiation (per-agent identity, signed delegation chains, tamper-evident logs — Baselines 2–4) do not close the gap, and to name the standardized-but-under-emphasized layer hypothesized to close it. Measuring Baseline 5 against the same attack is the primary defensive item of future work. Naming it converts the strongest RFC-literate objection — “you ignored token binding” — into a scope boundary chosen on purpose.

The discipline that protects all of the above.

Every AIS value in §6 is a pre-registered hypothesis, stated in this threat model before any attack code is written. If measurement contradicts a prediction — Baseline 3 not fully collapsing, Baseline 2 not reaching 1.0 — the discrepancy is reported as a finding,

not quietly reconciled. This is the structural difference between a benchmark and a demo: the predictions are committed in advance, so the result means something whether or not it confirms the hypothesis.
